

# Laboratorio Experiencia Físico-Digital

## Piloto de nave/dron

Andrés Botero, Jade Moreno, Sofía Reina y Vivi Delgado

**Resumen**—En este trabajo se desarrolló una experiencia físico-digital mediante la integración de un microcontrolador Arduino Pro Micro con una aplicación interactiva desarrollada en Godot. Para la construcción del controlador físico se utilizó una placa perforada, sobre la cual se ensamblaron un joystick analógico y tres pulsadores. Las señales generadas por estos componentes fueron capturadas por el microcontrolador y transmitidas hacia el entorno digital mediante una conexión USB. Los datos recibidos fueron procesados en Godot y utilizados para controlar una experiencia interactiva de temática espacial, en la cual el usuario maneja una nave e interactúa con diferentes asteroides. Los resultados obtenidos durante las pruebas permitieron comprobar el funcionamiento estable de la comunicación entre el controlador físico y la aplicación, demostrando la viabilidad del uso de Arduino y USB como medio de interacción para experiencias digitales desarrolladas en Godot.

**Abstract**—In this work, a physical-digital experience was developed by integrating an Arduino Pro Micro microcontroller with an interactive application built in Godot. To build the physical controller, a perforated board was used, on which an analog joystick and three push buttons were mounted. The signals generated by these components were captured by the microcontroller and transmitted to the digital environment over a USB connection. A basic validation of the messages was performed, checking that they matched the established format and discarding those that did not. The results obtained during testing confirmed the stable operation of the communication between the physical controller and the application, demonstrating the feasibility of using Arduino and USB as an interaction medium for digital experiences developed in Godot.

**Index Terms**—Arduino Pro Micro, Godot, comunicación USB, interacción físico-digital, joystick analógico, microcontrolador, transmisión de datos.

### I. INTRODUCCIÓN

La integración entre dispositivos físicos y entornos digitales permite desarrollar experiencias interactivas en las que las acciones realizadas mediante un controlador físico pueden ser interpretadas y representadas dentro de una aplicación. En este contexto, los microcontroladores permiten adquirir señales provenientes de diferentes dispositivos de entrada y transmitirlas hacia un entorno de software, estableciendo una comunicación entre el hardware y la experiencia digital.

En el presente laboratorio se desarrolló una experiencia físico-digital mediante la integración de un microcontrolador Arduino Pro Micro con una aplicación interactiva desarrollada en Godot. Para la construcción del controlador físico se utilizó una placa perforada, sobre la cual se ensamblaron los diferentes componentes de entrada. El sistema incorpora un joystick analógico y tres pulsadores, cuyos valores y estados son capturados por el microcontrolador y transmitidos hacia el entorno digital mediante una conexión USB.

Para lograr esta comunicación fue necesario establecer una estructura para los mensajes enviados desde el Arduino Pro Micro hacia Godot y realizar una validación básica de los datos recibidos. Los mensajes utilizados durante las pruebas presentan un tamaño inferior a 20 bytes, por lo que su impacto sobre el rendimiento de la comunicación es reducido. Debido a que la transmisión de información se realiza de manera continua, los mensajes que no cumplen con el formato establecido pueden ser descartados, permitiendo que los siguientes datos recibidos mantengan el funcionamiento de la interacción.

La utilización de USB como medio de comunicación permite simplificar la integración entre el controlador físico y la aplicación, evitando configuraciones adicionales o dependencias externas para establecer la comunicación. De esta manera, el desarrollo permite aplicar conceptos relacionados con la adquisición de señales, transmisión de datos y comunicación entre hardware y software en una experiencia interactiva desarrollada en Godot.

### II. OBJETIVOS

#### II-A. Objetivo general

Implementar y evaluar una experiencia físico-digital basada en la comunicación entre un microcontrolador Arduino Pro Micro y un entorno interactivo en Godot.

#### II-B. Objetivos específicos

- Diseñar y construir un circuito físico basado en un microcontrolador Arduino Pro Micro, un joystick analógico y tres pulsadores para la captura de señales análogas y digitales.
- Investigar y aplicar los fundamentos técnicos de la comunicación USB nativa (HID) y protocolos seriales ligeros como Teleplot para la transmisión de comandos hacia entornos virtuales.
- Desarrollar la integración de las señales físicas con el simulador interactivo “Piloto de nave/dron” en Godot Engine, validando el comportamiento operativo bajo condiciones reales.
- Analizar el rendimiento de la comunicación en términos de tamaño de paquete, eficiencia del bus y estrategias de tolerancia a fallos mediante validación de formato.

### III. MARCO TEÓRICO

#### III-A. Estándares de Construcción de Paquetes de Datos (Framing)

En el contexto de las comunicaciones seriales, el flujo de información transmitida debe estructurarse adecuadamente para

que el sistema receptor pueda distinguir de manera inequívoca el principio y el final de cada mensaje [1]. Para lograr este propósito, existen diferentes estrategias de empaquetado o *framing*:

- **Longitud fija frente a estructuración por delimitador:** Un enfoque común es establecer tramas de longitud fija, donde el receptor conoce de antemano exactamente cuántos bytes componen un paquete. Esta técnica minimiza la sobrecarga computacional relacionada con el análisis sintáctico del mensaje, pero resulta ineficiente si el tamaño de los datos varía frecuentemente, ya que obliga a rellenar con bytes vacíos desperdiciando el ancho de banda del canal [2]. Por otro lado, la estructuración por delimitador flexibiliza la longitud del paquete al utilizar caracteres especiales (conocidos como banderas o *flags*) para señalar los bordes de la trama. Esto optimiza el uso del canal de comunicación, aunque exige que el decodificador escanee de forma continua el flujo de entrada en busca de dichos marcadores [1].
- **Uso de encabezados (Headers):** Es una práctica extendida que los paquetes estructurados incluyan un bloque inicial llamado encabezado, cuya función principal es proporcionar metadatos cruciales sobre la carga útil (*payload*). Dicho encabezado suele especificar parámetros como la longitud exacta de los datos adjuntos, el tipo de instrucción contenida o las direcciones lógicas de la red, facilitando así la preparación de buffers en memoria y el procesamiento por las capas superiores antes de que el paquete llegue completo [2].
- **Byte Stuffing (Relleno de bytes y Escaping):** La adopción de delimitadores introduce un desafío técnico crítico en la transmisión de datos crudos: si el byte empleado como bandera delimitadora aparece accidentalmente dentro de la propia carga útil, el sistema receptor cerrará y fragmentará la trama de manera prematura. Para mitigar esta colisión de datos, se emplea la técnica de *byte stuffing* (relleno de bytes) o el uso de secuencias de escape. Este procedimiento consiste en insertar un carácter especial (como un byte nulo o una barra invertida) inmediatamente antes de la coincidencia conflictiva, indicándole al módulo de recepción que debe tratar el siguiente byte puramente como información y no como una instrucción de control [1].

### III-B. JSON como Formato de Intercambio y Deserialización en Entornos Interactivos

JSON (JavaScript Object Notation) se ha consolidado como un estándar universal para la transferencia de información entre sistemas gracias a que es un formato de texto plano, altamente ligero y fácilmente interpretable. Al integrar interfaces de hardware físico, como microcontroladores, con entornos simulados, el proceso algorítmico encargado de traducir estas cadenas de texto continuo en estructuras de datos utilizables se conoce como deserialización. Dado que la construcción de ecosistemas interactivos puede abordarse desde distintos motores gráficos, el enfoque de deserialización varía sustancialmente según la arquitectura del software:

- **Deserialización en C# (Unity):** En el contexto de Unity, el proceso requiere mapear el texto a objetos fuertemente tipados. El motor provee `JsonUtility`, una biblioteca nativa implementada en C++ que garantiza tiempos de procesamiento excepcionalmente rápidos y una asignación de memoria mínima, evitando invocar frecuentemente al recolector de basura (*garbage collector*) durante ciclos de actualización rápidos [3]. No obstante, presenta restricciones al no admitir la deserialización directa de diccionarios o arreglos complejos [3]. Para solventar estas limitaciones, se recurre a `Newtonsoft.Json (Json.NET)`, una biblioteca de terceros que proporciona una libertad estructural profunda para evaluar datos dinámicos, aunque introduce una mayor sobrecarga computacional [4].
- **Deserialización nativa en Godot Engine:** Como contraparte y herramienta implementada en el desarrollo práctico de este laboratorio, Godot Engine aborda el procesamiento de datos bajo un paradigma de tipado dinámico, especialmente al emplear su lenguaje principal, GDScript. Godot resuelve la deserialización sin depender de bibliotecas externas de terceros mediante su clase global `JSON` [5]. A diferencia de Unity, donde se requiere la declaración previa de clases contenedoras, el método `JSON.parse_string()` de Godot interpreta la cadena serializada y la convierte de manera inmediata en una estructura de datos dinámica, usualmente un objeto de tipo *Dictionary* o *Array* [5]. Si bien esto agiliza el desarrollo iterativo y reduce la verbosidad del código, la ausencia de un esquema rígido tipado transfiere la responsabilidad de la validación de integridad (como la verificación de la existencia de claves específicas) directamente a la lógica del desarrollador [5].

### III-C. Métodos de Comprobación de Integridad de Datos

Dado que los canales de comunicación serial están frecuentemente expuestos al ruido eléctrico y a las interferencias electromagnéticas, es indispensable implementar métodos matemáticos que permitan al receptor validar si los bytes transferidos sufrieron alteraciones durante el recorrido [1]:

- **Bit de paridad:** Representa el mecanismo de verificación más rudimentario. Se basa en concatenar un bit redundante al final de la palabra de datos con el objetivo de forzar que el conteo total de unos lógicos (1) sea par o impar. Si bien su costo computacional es prácticamente inexistente en microcontroladores, su nivel de robustez es deficiente; este método falla sistemáticamente ante ráfagas de error, ya que no puede alertar anomalías si un número par de bits cambia de estado de manera simultánea [2].
- **Checksum (Suma de comprobación módulo 256 y XOR):** Involucra ejecutar operaciones algebraicas elementales sobre el paquete. Por ejemplo, consiste en realizar la suma (descartando el desbordamiento de 8 bits) de todos los bytes del mensaje, o aplicar la compuerta lógica XOR de forma acumulativa sobre la secuencia, anexando el resultado como firma final de la trama. El dispositivo receptor repite el cálculo para corroborar las firmas. Estas técnicas consumen recursos computacionales muy bajos,

haciéndolas ideales para el hardware embebido [2]. Su robustez es considerada de nivel medio, ya que en algunos casos puede fallar cuando la corrupción de datos sigue algunos patrones específicos [1].

- **CRC (Comprobación de Redundancia Cíclica):** Es un enfoque polinomial que ofrece alta precisión. Analiza la totalidad del bloque de bits asumiendo que es un coeficiente polinómico gigante, el cual se divide iterativamente (mediante aritmética XOR) por una constante matemática conocida como polinomio generador; el residuo resultante de esta división se adjunta al mensaje [1]. El CRC destaca por una robustez excepcional, logrando detectar alteraciones de ráfagas largas e inversiones complejas que el *checksum* omitiría [2]. La desventaja de este algoritmo es su costo computacional, el cual es el más alto entre las opciones. En general requiere de optimizaciones específicas para que no sea el cuello de botella en la ejecución del firmware [1].

### III-D. Protocolos de Enlace Físico: UART y USB

Los protocolos de enlace definen la forma en que los bits son transmitidos a través de un medio físico y constituyen la base sobre la cual se construyen las tramas descritas en las secciones anteriores [1]. En el desarrollo de este laboratorio conviven dos tecnologías de enlace con filosofías marcadamente distintas: la comunicación serial asíncrona UART y el bus USB.

- **UART (Universal Asynchronous Receiver-Transmitter):** Es un protocolo serial **asíncrono**, lo que implica que no existe una línea de reloj compartida entre el emisor y el receptor; ambos extremos deben acordar previamente la velocidad de transmisión (*baud rate*) y el formato de cada carácter [2]. Cada palabra se delimita mediante un bit de arranque (*start bit*), seguido de los bits de datos (habitualmente 8), un bit de paridad opcional y uno o más bits de parada (*stop bit*). La conexión es punto a punto y **full-duplex**, lo que significa que siempre se pueden enviar mensajes en las dos direcciones y que los mensajes que viajan en una dirección no bloquean a los que viajan en la otra. UART no define por sí mismo una estructura de paquete a nivel de mensaje: al operar sobre un flujo continuo de bytes, tanto la delimitación de tramas (por delimitador o por longitud fija) como la verificación de integridad deben implementarse en capas superiores [1].
- **USB (Universal Serial Bus):** A diferencia de UART, USB es un bus **centralizado** y **sondeado** (*polled*): existe un único anfitrión (*host*) que administra el medio, al cual se conectan todos los dispositivos. Al conectar un dispositivo, el anfitrión ejecuta un proceso de **enumeración** en el que le asigna una dirección y le pregunta cuáles son sus capacidades. El dispositivo responde con una **tabla de descriptores**, es decir, tablas que le indican al anfitrión qué datos puede emitir y con qué estructura lo hace [6].

**La ventaja del Arduino Pro Micro:** Esta placa integra la función USB en su propio microcontrolador (ATmega32U4). Esto significa que el chip principal expone directamente la comunicación serial como una de sus capacidades USB, sin necesidad de un chip adicional. Además, permite programar el

dispositivo para exponer otras capacidades USB, como HID (mouse, teclado o control de juego). De esta manera es posible usar la misma conexión para transportar tanto la comunicación serial como las entradas del controlador [7].

### III-E. Estructura de los Paquetes USB HID

A diferencia de los protocolos de texto, que delimitan cada mensaje de forma explícita, el perfil **HID (Human Interface Device)** de USB describe sus datos una sola vez y luego los envía siempre con el mismo formato [6]. Esa descripción se realiza mediante el **descriptor de reporte** (*Report Descriptor*), que funciona como una cabecera: primero establece un identificador (*Report ID*) y enseguida describe qué datos van a llegar y con qué formato (por ejemplo, cuántos bits ocupa cada campo). Esta cabecera se declara durante la enumeración y no se repite en cada mensaje.

Así, cada **paquete de reporte** (*Input Report*) es simplemente la carga útil enviada: una secuencia de bits con los campos ya definidos en el descriptor. Como el anfitrión conoce de antemano el formato, el reporte no necesita encabezado, longitud ni firmas propias, lo que da máxima eficiencia y una latencia predecible, a cambio de no poder cambiar el formato sin volver a declarar el descriptor.

## IV. METODOLOGÍA Y DESARROLLO

### IV-A. Descripción del Montaje Físico y Hardware

El desarrollo del mecanismo físico-digital se implementó con un microcontrolador **Arduino Pro Micro**. La elección del Arduino Pro Micro se fundamenta en su capacidad nativa de emulación de dispositivos USB HID, lo que simplifica la transmisión de datos hacia el entorno digital.

El control construido consta de una palanca de dos ejes (X e Y), que se comporta como dos potenciómetros independientes, y tres pulsadores (A, B y C). Además, la propia palanca incorpora un cuarto pulsador (D) que se activa al presionarla hacia abajo. Los ejes entregan valores analógicos en un rango de 0 a 1023, mientras que los pulsadores entregan valores digitales. El montaje se hizo con soldadura sobre una placa perforada para asegurar la estabilidad de las conexiones, como se muestra en la Figura 1.

### IV-B. Protocolos de Comunicación Implementados

Para comunicar nuestros sensores físicos con Godot implementamos varios protocolos de comunicación. Algunos usan el puerto serial, y otros usan la interfaz USB HID.

Conviene aclarar la intención detrás de estas elecciones. No nos limitamos a una lista cerrada de opciones: adoptamos **Teleplot** como una alternativa conveniente que encontramos para poner a prueba el circuito de forma rápida y sencilla, y que asimismo resulta útil para depurar el producto final. USB HID y JSON se sumaron después como representantes de un bus nativo del hardware y de un formato de intercambio estándar, respectivamente, para poder comparar enfoques distintos.

- **Variables Estructuradas:** En principio usamos el protocolo que usa **Teleplot** para validar el circuito de manera fácil antes de implementar cualquier cosa en Godot. Este

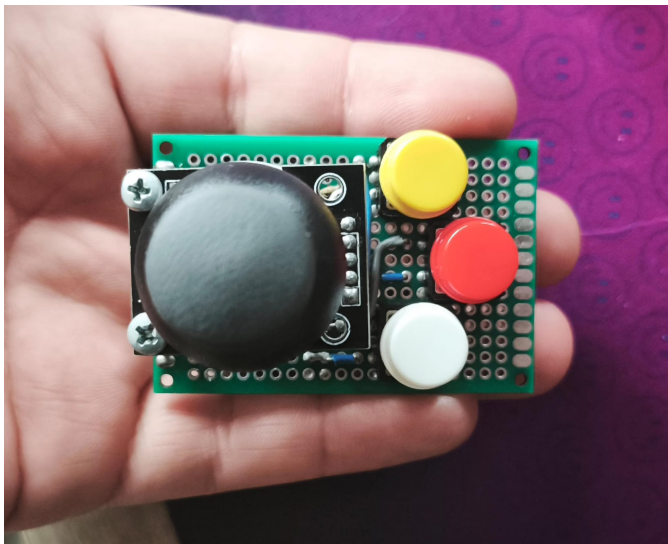


Figura 1. Montaje físico del controlador: joystick analógico de dos ejes y tres pulsadores soldados sobre una placa perforada.

protocolo está orientado a la transmisión estructurada de paquetes de telemetría y comandos de control a través del puerto serial.

En el código generamos mensajes en formato ASCII usando los identificadores A, B, C y D para los botones, y los identificadores X, Y para los ejes analógicos. Se emite un mensaje por cada botón o sensor, y los mensajes siguen un formato del tipo >VARIABLE:VALOR, usando un salto de línea como delimitador entre ellos.

Este protocolo nos permite visualizar gráficamente la variación de las entradas durante la manipulación del controlador físico usando el plugin de Teleplot en Visual Studio Code, como se muestra en la Figura 2. De esta manera pudimos validar la correcta instalación del circuito antes de proceder con la implementación de los otros protocolos.

Adicionalmente, durante una de estas pruebas notamos una conexión cruzada entre las señales de los botones y los potenciómetros. Este fallo se manifestaba como una caída en las lecturas de los potenciómetros cuando un botón estaba presionado. Esta validación nos llevó a retrabajar la placa antes de continuar con el desarrollo del proyecto.

- **USB HID (Human Interface Device):** Se configuró directamente en el firmware del Arduino Pro Micro para que el microcontrolador se comporte como un periférico de juego estándar. Mediante este mecanismo, el controlador físico puede transmitir las entradas del joystick y los botones como eventos de un dispositivo HID. El firmware define un reporte de tipo gamepad, con campos para los botones y los ejes X y Y, permitiendo que el sistema operativo y el entorno de desarrollo reconozcan el dispositivo como un controlador de entrada.

En los reportes que se envían, el dispositivo declara soportar todos los botones de un gamepad durante la enumeración, aunque solamente usa unos bits específicos



Figura 2. Visualización de las señales del controlador físico mediante el plugin Teleplot en Visual Studio Code.

para los cuatro pulsadores del dispositivo. Estos se reportan con los usos (face down, face right, face up, left thumbstick) [7], [6].

Este enfoque simplifica la integración con Godot, ya que las acciones del usuario se pueden interpretar como acciones de cualquier control de juegos.

El Listado 1 presenta una captura de los eventos emitidos por el dispositivo, tal como los registra la herramienta `evtest` en Linux. Allí se observan los reportes de los ejes analógicos (ABS\_X y ABS\_Y) y de los pulsadores (BTN\_EAST), separados por los eventos de sincronización SYN\_REPORT:

```
Event: time 1791164627.592949, -----
      SYN_REPORT -----
Event: time 1791164627.593950, type 3 (EV_ABS),
      code 0 (ABS_X), value 507
Event: time 1791164627.593950, type 3 (EV_ABS),
      code 1 (ABS_Y), value 512
Event: time 1791164627.599949, -----
      SYN_REPORT -----
Event: time 1791164628.926961, type 4 (EV_MSC),
      code 4 (MSC_SCAN), value 90002
Event: time 1791164628.926961, type 1 (EV_KEY),
      code 305 (BTN_EAST), value 0
Event: time 1791164628.926961, -----
      SYN_REPORT -----
Event: time 1791164629.008966, type 4 (EV_MSC),
      code 4 (MSC_SCAN), value 90002
Event: time 1791164629.008966, type 1 (EV_KEY),
      code 305 (BTN_EAST), value 1
```

Listing 1. Eventos HID reportados por el Arduino Pro Micro, capturados con `evtest` en Linux.

- **Representación en JSON:** Estándar de intercambio basado en texto plano estructurado mediante objetos clave-

valor. Nuestra implementación de JSON es deliberadamente simple: codifica el mismo conjunto fijo de variables (los pulsadores y los ejes) en un solo objeto por mensaje, sin estructuras anidadas ni tipos de dato variados. Esa simplicidad es suficiente para las necesidades actuales y favorece la legibilidad, pero también implica que cualquier requerimiento más avanzado —estructuras anidadas, tipos de dato distintos o mensajes cuyo formato cambie en tiempo de ejecución— exigiría una solución más compleja, tanto en la codificación como en la deserialización. Como todos nuestros valores son enteros y no contienen saltos de línea, el separador de mensajes no introduce ambigüedades por ahora.

Captura de un mensaje emitido en formato JSON:

```
{ "T": 79659896, "A": 0, "B": 0, "C": 0,
  "D": 0, "X": 518, "Y": 506 }
```

Respecto a la integridad de los datos, la solución implementada no incorpora mecanismos explícitos como *checksum* o CRC, y esa decisión se sostiene en dos razones. Primero, la transmisión es continua y redundante: cada muestra repite el estado completo del controlador, de modo que un mensaje malformado puede descartarse sin perder información, ya que el siguiente llega pocos milisegundos después. Sobre esa base, el código en Godot valida cada mensaje contra la estructura esperada y descarta los que no la cumplen, sin intentar reconstruirlos ni corregirlos. Segundo, en el caso de USB HID la integridad se delega al propio protocolo del bus: USB verifica cada transacción con sus mecanismos propios y, como el Arduino Pro Micro se presenta como un gamepad estándar, se asume que el transporte funciona correctamente y no se añade verificación en la aplicación.

#### IV-C. Implementación de los controles en Godot Engine

El desarrollo del software interactivo se implementó en **Godot Engine**, empleando su arquitectura de nodos y scripts en GDScript para gestionar la simulación de los cuerpos físicos. El taller sugería Unity como motor; sin embargo, tomamos la libertad de usar Godot, un motor de código abierto con el que el grupo ya estaba familiarizado. Esta decisión no afecta el objeto de estudio, ya que la comparación entre protocolos de comunicación es independiente del motor gráfico empleado. Todo el código fuente del sistema, incluyendo los scripts de control y la estructura del proyecto, se encuentra disponible en el repositorio oficial del grupo [9].

Para los protocolos que usaban comunicación serial directamente, usamos el plugin externo GdSerial para leer el dispositivo serial [8]. Dentro del proyecto se puede configurar si el formato de los mensajes es el de Teleplot (lo llamamos DEBUG) o si es formato JSON. De esta manera, el código elige qué camino tomar al momento de interpretar los mensajes.

Para cualquiera de los casos, sea formato Teleplot o JSON, el código genera la misma señal de `input_received` a la cual otros sistemas del motor pueden conectarse para reaccionar a los eventos recibidos del Arduino. Adicionalmente, esta señal emite datos de tipo `InputEvent`, similar a como hace Godot con sus eventos nativos, lo que permite manejarlos de forma sencilla.

En el caso de USB HID, no hizo falta escribir código adicional para el manejo de la comunicación, ya que todos los eventos son capturados automáticamente por el motor, que emite los callbacks a la función `_input` por defecto.

Para unificar el funcionamiento y simplificar el código, tanto los mensajes obtenidos por el puerto serial (recibidos mediante la señal `input_received`) como los mensajes USB HID (recibidos mediante el llamado a `_input`) hacen el mismo llamado a la función `on_input_received`, lo que nos permite controlar la nave de la misma manera, sin importar de dónde llegó el mensaje.

Como todos los caminos del código se unifican al final y reciben eventos con la estructura nativa del motor, es posible usar otros mecanismos como `InputMap` para facilitar la implementación de la lógica de la simulación.

#### IV-D. Desarrollo e integración de la experiencia interactiva

Como entorno de simulación interactiva se nos asignó el escenario **Piloto de nave/dron**, el cual exige un control continuo de la altitud y aceleración mediante los ejes del joystick, complementado por los pulsadores para funciones de disparo, impulso y alternancia de modos. Este escenario es relevante para el estudio porque combina entradas continuas —los dos ejes del joystick, que varían de forma constante— con entradas discretas —los pulsadores—. Esa mezcla obliga a que la comunicación sostenga una actualización fluida y una latencia baja, por lo que permite comparar los protocolos bajo una carga representativa del control en tiempo real.

La aplicación desarrollada en Godot consiste en una experiencia interactiva de temática espacial en la que el usuario controla una nave y debe interactuar con diferentes asteroides. El usuario maneja la simulación con el controlador conectado al Arduino, y sus acciones son interpretadas por el sistema de entrada de Godot mediante las acciones definidas en el *Input Map*. Estas entradas permiten controlar el movimiento de la nave y ejecutar acciones como disparar, impulsar y girar, estableciendo una conexión directa entre la manipulación del controlador físico y la respuesta dentro de la experiencia digital. La Figura 3 muestra la experiencia en ejecución.

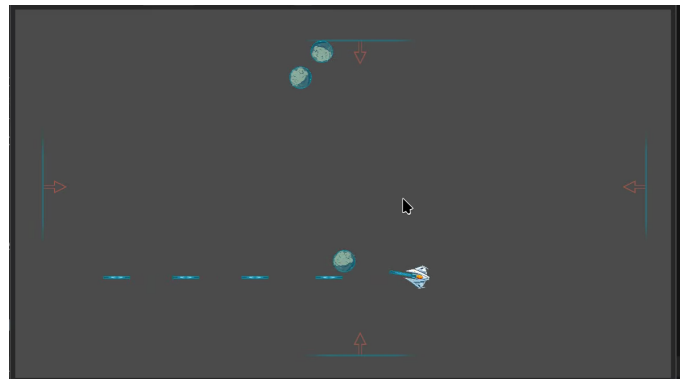


Figura 3. La experiencia interactiva en ejecución: la nave dispara a los asteroides en el escenario Piloto de nave/dron.

En el planteamiento del ejercicio se consideraba controlar una simulación con control de altitud y aceleración, que

son conceptos complejos de la física de un dron. Como el enfoque de la actividad es el desarrollo de los protocolos de comunicación, solo se implementó una física simple, donde la entrada del jugador se traduce directamente en la velocidad de la nave, sin conceptos de gravedad o aceleración. El escenario de simulación implementado es una réplica sencilla del juego Asteroids.

#### IV-E. Procedimiento de Medición

Se definieron criterios cualitativos y cuantitativos. Para evaluar el desempeño de las implementaciones se estableció un procedimiento experimental basado en las siguientes métricas:

1. **Latencia de transmisión:** el intervalo de muestreo (el periodo con que el controlador produce cada estado) acota el tiempo mínimo de reacción, por lo que se midió como indicador de latencia.
2. **Legibilidad y depuración:** evaluación cualitativa de la facilidad para diagnosticar errores mediante el monitor serial y la traza de eventos.
3. **Costo de procesamiento:** tamaño de la muestra en bytes y ancho de banda resultante.

Para las métricas cuantitativas se desarrolló, con ayuda de un asistente de inteligencia artificial (GitHub Copilot), un arnés de medición en Python (`tools/medir_serie.py` y `tools/medir_hid.py`) que registra cada mensaje recibido con una marca de tiempo de alta resolución. La unidad de medida es la *muestra*, entendida como un estado completo del controlador: en Teleplot son siete líneas, en JSON una línea y en USB HID un reporte. Se midieron dos escenarios de aproximadamente nueve segundos cada uno: **tasa máxima** (el firmware envía en cada vuelta del bucle) y **tasa fija** de 200 muestras por segundo, fijada con `SEND_EVERY_MS`. De cada corrida se registraron el número de muestras, los bytes por muestra, la tasa y dos intervalos: el medido en el host (entre llegadas consecutivas) y el del firmware (a partir de su marca de tiempo  $T$ ), que no depende del render de Godot ni del búfer del host. Para USB HID, cuyos reportes idénticos pueden no generar evento en la capa de entrada del sistema operativo, se usó además un **latido** que alterna un botón no usado para forzar que cada reporte cambie.

## V. RESULTADOS

La comparación de las tres implementaciones se organizó en dos dimensiones complementarias: una cualitativa, que recoge la experiencia del equipo al implementar, integrar y depurar cada protocolo, y una cuantitativa, que reúne métricas objetivas sobre el costo y el ritmo de transmisión de los mensajes.

#### V-A. Comparación cualitativa

La Tabla I resume la evaluación cualitativa de los tres protocolos según cuatro criterios: la facilidad de implementación en el Arduino, la facilidad de integración en Godot, la facilidad para leer los mensajes crudos y otras ventajas de cada enfoque.

Tabla I  
COMPARACIÓN CUALITATIVA DE LAS TRES IMPLEMENTACIONES.

| Criterio                               | Teleplot           | USB HID          | JSON             |
|----------------------------------------|--------------------|------------------|------------------|
| Facilidad de implementación en Arduino | Alta               | Media            | Alta             |
| Facilidad de integración en Godot      | Media              | Alta             | Media            |
| Facilidad para leer mensajes crudos    | Alta               | Baja             | Alta             |
| Otras ventajas                         | Graficación nativa | Gamepad estándar | Formato conocido |

#### V-B. Comparación cuantitativa

La Tabla II presenta las métricas cuantitativas. El tamaño de payload corresponde a los bytes útiles por muestra; el intervalo de envío es el periodo con que el firmware produce cada muestra, y de él se deriva la tasa estimada ( $1/\Delta t$ ). La tasa medida corresponde al conteo real de muestras recibidas por el host.

Tabla II  
COMPARACIÓN CUANTITATIVA DE LAS TRES IMPLEMENTACIONES.

| Métrica                             | Teleplot | USB HID | JSON  |
|-------------------------------------|----------|---------|-------|
| Tamaño de payload (bytes)           | 53       | 6       | 56    |
| <i>Tasa fija (200 muestras/s)</i>   |          |         |       |
| Tasa medida (muestras/s)            | 702.9    | 1000.0* | 617.7 |
| Intervalo entre muestras (ms)       | 1.426    | 1.000   | 1.623 |
| <i>Tasa máxima (envío continuo)</i> |          |         |       |
| Tasa medida (muestras/s)            | 199.4    | 199.2   | 199.3 |
| Intervalo entre muestras (ms)       | 5.019    | 5.020   | 5.019 |
| Jitter del intervalo (desv., ms)    | 0.32     | 0.47    | 0.32  |
| Ancho de banda (kB/s)               | 10.6     | 1.2     | 11.1  |
| Retardo extra 1-vía, mediana (ms)   | 0.57     | —       | 0.54  |

El intervalo entre muestras proviene de la marca de tiempo  $T$  del firmware en los protocolos seriales y de los *timestamps* del kernel en USB HID. En el escenario de **tasa máxima**, Teleplot y JSON alcanzaron 702.9 y 617.7 muestras/s, mientras que USB HID midió 453.7 muestras/s sin forzar cambios: la capa de entrada del sistema operativo no genera evento para los reportes cuyo estado no cambia y solo registra las variaciones. Al forzar cada reporte a cambiar mediante el latido (\*), USB HID alcanzó **1000.0 muestras/s**, el máximo impuesto por el sondeo USB de 1 ms, por encima de los protocolos seriales. En el escenario de **tasa fija** los tres protocolos convergen en 199.2–199.4 muestras/s, exactamente el valor fijado, lo que confirma que ninguno constituye un cuello de botella para el control en tiempo real. El jitter del intervalo es sub-milisegundo en los tres casos ( $\sigma \leq 0.47$  ms), y el retardo adicional de un solo sentido tiene una mediana inferior a 0.6 ms, despreciable frente a los 16.7 ms de un cuadro a 60 cuadros por segundo. USB HID transmite alrededor de nueve veces menos datos por muestra (6 B frente a 53–56 B), a cambio de depender del sondeo del bus.

Para que las mediciones del lado del receptor no dependan del ciclo de renderizado, la captura se realizó con un arnés independiente de Godot y el intervalo se calcula a partir de la marca de tiempo ( $T$ ) que el firmware incluye en cada muestra, y no del instante en que el motor dibuja el cuadro. El intervalo

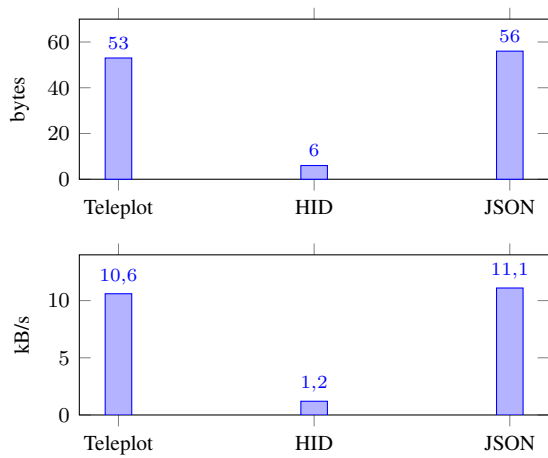


Figura 4. Tamaño de payload (arriba) y ancho de banda a tasa fija de 200 muestras/s (abajo) por protocolo.

medido en el host presentó valores extremos (de hasta 100 ms) por la entrega en ráfagas del bus USB, por lo que se prioriza el intervalo del firmware.

## VI. ANÁLISIS Y DISCUSIÓN

Los resultados obtenidos muestran que las tres implementaciones evaluadas —Teleplot, USB HID y JSON— cumplen con el objetivo de transportar las señales del controlador físico hacia el simulador: en todas ellas el usuario logró manejar la nave de la misma manera y la respuesta percibida fue equivalente. Las diferencias observadas en los tres criterios medidos (latencia, legibilidad y costo de procesamiento) resultaron lo suficientemente pequeñas como para no ser un factor decisivo al momento de seleccionar un protocolo. Esto es coherente con lo esperado según el marco teórico, ya que cada actualización transporta una cantidad de información muy reducida —el estado de cuatro pulsadores y dos ejes analógicos—, muy por debajo de la capacidad tanto del puerto serial como del enlace USB.

En cuanto a la legibilidad, Teleplot y JSON tienen la ventaja de ser texto plano: los mensajes pueden inspeccionarse directamente en el monitor serial y permiten depurar el circuito sin herramientas adicionales. Sin embargo, esa legibilidad tiene un costo, pues el receptor debe separar y convertir los valores (y, en el caso de JSON, deserializar la cadena), y tanto la delimitación de las tramas como la validación de integridad recaen sobre la capa de aplicación, tal como ocurre con UART [1]. USB HID, en cambio, describe su formato una sola vez mediante el descriptor de reporte y luego envía siempre la misma carga útil, sin encabezado, longitud ni firmas en cada mensaje, y sin necesidad de separadores ni de escape de caracteres [6].

En la práctica, la conveniencia de utilizar USB HID resultó superior a la flexibilidad o al rendimiento de las alternativas por puerto serial, al menos para este caso de uso específico, en el que el microcontrolador se emplea únicamente para capturar las entradas del usuario. Al presentar el dispositivo como un gamepad estándar, el sistema operativo y el motor (Godot) reconocen el controlador de forma automática, sin requerir

código adicional de comunicación, ya que la interpretación de los reportes queda a cargo del stack USB del anfitrión. Esto elimina de raíz problemas propios de los protocolos de texto, como la fragmentación de los mensajes, la pérdida de sincronía cuando un mensaje llega incompleto o la ambigüedad al interpretar un separador presente dentro de los datos. Para un dispositivo dedicado a reportar unas pocas entradas, la estructura fija de HID evita trabajo y reduce puntos de fallo sin sacrificar latencia.

No obstante, esta conclusión está acotada al escenario del laboratorio. Si en etapas posteriores el Arduino debiera enviar información más variada —por ejemplo, telemetría de varios sensores con estructuras distintas, o mensajes cuyo formato cambie en tiempo de ejecución—, la flexibilidad de un protocolo serial con delimitación explícita, o de un formato como JSON, resultaría más adecuada, ya que HID obliga a declarar el formato de antemano y a rediseñar el descriptor si este cambia. Por ello, la elección de USB HID debe entenderse como la mejor opción para el caso específico evaluado, y no como una regla general.

## VII. CONCLUSIÓN

El objetivo general de este laboratorio era implementar y evaluar una experiencia físico-digital basada en la comunicación entre un microcontrolador Arduino Pro Micro y un entorno interactivo en Godot. Los resultados obtenidos permiten afirmar que dicho objetivo se cumplió: el controlador físico y el simulador se comunicaron de manera estable y el usuario pudo interactuar con la experiencia de forma fluida.

Cada objetivo específico planteado al inicio encontró también su cierre en el desarrollo del trabajo. Se diseñó y construyó el circuito con el Arduino Pro Micro, el joystick analógico y los pulsadores (Figura 1), logrando la captura correcta de las señales analógicas y digitales. Sobre esa base se investigaron y aplicaron los fundamentos de la comunicación USB nativa (HID) y de los protocolos seriales ligeros como Teleplot, lo que dio lugar a tres implementaciones —Teleplot, USB HID y JSON— comparadas bajo las mismas condiciones. Cada una de ellas se integró con el simulador asignado, Piloto de nave/dron, validando su funcionamiento en tiempo real, y finalmente se analizó su desempeño en términos de tamaño de paquete, eficiencia del bus y tolerancia a fallos mediante la validación del formato.

El análisis de esas tres alternativas nos demuestra que la mejor herramienta depende siempre del problema que queramos solucionar, y no de una preferencia técnica fija. Para el caso estudiado —un dispositivo dedicado únicamente a capturar las entradas del usuario— USB HID resultó la opción más conveniente, gracias a su integración automática con el sistema operativo y el motor, que evita el trabajo de análisis y validación en la aplicación. En cambio, si el sistema requiriera transmitir información más variada o con formato cambiante, la flexibilidad de un protocolo serial como Teleplot o JSON sería preferible.

Conviene precisar el alcance de la recomendación, porque este laboratorio y el **proyecto del vehículo** plantean problemas distintos. Aquí el reto era acotado —capturar las entradas de

un controlador físico—, y para ese problema USB HID fue la mejor opción. El **proyecto del vehículo** requiere comunicación inalámbrica y protocolos de Internet, para lo que usaremos un microcontrolador **ESP32** con **TCP** o **UDP**. En ese escenario no existe un perfil equivalente a HID, de modo que la solución exigirá **procesamiento manual de los mensajes** (construcción de tramas, delimitación y validación). Ahí es donde este laboratorio deja un aporte reutilizable: el manejo de datos desarrollado para el puerto serial —codificación, delimitación, validación de formato y descarte de mensajes malformados— sirve como base para procesar los mensajes sobre TCP/UDP.

En síntesis, lo verdaderamente importante no es memorizar un protocolo específico, sino conocer las herramientas disponibles y comprender sus ventajas y limitaciones para poder tomar una decisión informada según los requisitos de cada proyecto. Ese criterio, aprendido a lo largo de esta exploración, será el que guíe la selección de la tecnología de comunicación en las siguientes etapas del curso.

## VIII. ENLACE AL REPOSITORIO DE GITHUB

El código fuente completo del proyecto —el firmware del Arduino (proyecto PlatformIO) y la experiencia interactiva en Godot— está disponible públicamente en el repositorio oficial del grupo:

[https://github.com/botero-dev/usbbog\\_realidadmixta\\_lab](https://github.com/botero-dev/usbbog_realidadmixta_lab)

La demostración del sistema en funcionamiento está en el video `informe/assets/simulador.webm`, disponible en [https://github.com/botero-dev/usbbog\\_realidadmixta\\_lab/blob/main/informe/assets/simulador.webm](https://github.com/botero-dev/usbbog_realidadmixta_lab/blob/main/informe/assets/simulador.webm) (enlace tentativo, sujeto a la ubicación final del archivo).

## REFERENCIAS

- [1] Tanenbaum, A. S. y Wetherall, D. J. (2012). *Redes de computadoras* (5ª ed.). Pearson Educación. ISBN: 978-607-32-0817-8.
- [2] Stallings, W. (2004). *Comunicaciones y redes de computadores* (7ª ed.). Pearson Prentice Hall. ISBN: 84-205-4110-9.
- [3] Unity Technologies. (2024). *JSON Serialization*. Unity Documentation. Disponible en: <https://docs.unity3d.com/Manual/JSONSerialization.html>
- [4] Newton-King, J. (2024). *Json.NET Documentation*. Newtonsoft. Disponible en: <https://www.newtonsoft.com/json/help/html/Introduction.htm>
- [5] Godot Engine. (2024). *JSON class*. Godot Engine Documentation (Stable). Disponible en: [https://docs.godotengine.org/en/stable/classes/class\\_json.html](https://docs.godotengine.org/en/stable/classes/class_json.html)
- [6] USB Implementers Forum. (2006). *Device Class Definition for Human Interface Devices (HID), Version 1.11*. Disponible en: <https://www.usb.org/document-library/device-class-definition-hid-111>
- [7] Arduino. (2024). *Arduino Pro Micro Documentation*. Arduino Official Website. Disponible en: <https://www.arduino.cc/>
- [8] Christopher, S. (2026). *GdSerial: Serial Communication for Godot (v0.3.4)* [Complemento]. GitHub. Disponible en: <https://github.com/SujithChristopher/gdserial>
- [9] Botero, A. (2026). *Repositorio oficial del proyecto de Realidad Mixta: Experiencia Físico-Digital* [Código fuente]. GitHub. [https://github.com/botero-dev/usbbog\\_realidadmixta\\_lab](https://github.com/botero-dev/usbbog_realidadmixta_lab)